Frustration Signals in AI-Tutoring Transcripts: A Codebook and Detector for Programming Help Conversations

David Barron, Maya Torres, Jordan Lee

ISE 6664: Affective Design and Computing (Spring 2026) – Final Project Report

Instructor: Dr. Alan Brooks

May 2, 2026

Abstract

Students in introductory programming courses now ask AI assistants for help many times per assignment. Some of these conversations go well; others end with the student repeating the same question, pasting the same error, and giving up. We studied frustration in 180 de-identified student-assistant conversations from an IRB-approved CS1 lab study. We adapted an existing lexicon of frustration markers to programming help, separated frustration from confusion, and coded 2,912 student turns (Cohen's kappa 0.68 for frustration, 0.63 for confusion). Frustration rose sharply after the second consecutive failed run that followed an assistant suggestion, and the strongest single marker was repetition of a previous request with added negation ("no, it still doesn't work"). A detector built from marker counts reached AUC 0.74; normalizing counts against each student's own baseline raised it to 0.79, outperforming a zero-shot LLM classifier (0.71). We close with design implications for assistants that notice frustration and respond with acknowledgement, reframing and a change of strategy rather than another code block.

1 Introduction

AI programming assistants have become a routine part of how novices get help. A student who once waited in an office-hours queue now pastes an error message into a chat panel and receives an answer in seconds. Fast help is not always good help, however. Every TA has seen the pattern that motivated this project: a student asks for a fix, applies it, runs the code, sees a new error, asks again, and after a few rounds stops asking and stops working. In a human tutoring session the tutor would notice the slump in the student's voice or posture and change approach. A text assistant sees only the words.

Affective computing has long argued that systems which interact with people should be able to recognize and respond to emotion (Picard, 1997). In learning, the affective states that matter most are not the basic emotions but the learning-centered ones: confusion, frustration, boredom, engagement (D'Mello and Graesser, 2012; Baker et al., 2010). Frustration in particular is ambiguous. Some frustration accompanies productive struggle and resolves into learning; persistent frustration predicts disengagement.

This report asks three questions about frustration in conversations between CS1 students and an AI programming assistant:

• 1. RQ1: Which textual signals in student turns indicate frustration, and how do they differ from signals of confusion?

• 2. RQ2: When in a conversation does frustration appear, and what assistant behaviors precede it?

• 3. RQ3: How well can frustration be detected automatically from these signals, and does normalizing against each student's own baseline help?

Our contributions are (a) a codebook for frustration and confusion in programming help conversations, adapted from a validated tutoring lexicon (Albrecht and Nwosu, 2022); (b) a descriptive analysis of when frustration occurs; (c) a comparison of three detectors; and (d) design implications for assistants, grounded in affective design guidance.

2 Background

2.1 Affect in learning

Dimensional models describe affect along valence (pleasant to unpleasant) and arousal (calm to activated) (Russell, 1980). Frustration sits in the unpleasant, activated quadrant; boredom in the unpleasant, deactivated one. The distinction matters for learning. Research on learners' cognitive-affective states found that boredom was more persistent and more harmful to learning than frustration (Baker et al., 2010), which suggests that frustration is a state to manage, not necessarily to eliminate.

D'Mello and Graesser's account of affect dynamics during complex learning describes confusion as arising from impasses: if the impasse is resolved, the learner returns to engagement; if it is not, confusion can turn into frustration and eventually boredom (D'Mello and Graesser, 2012). This model shaped our coding scheme, because it treats confusion and frustration as related but separable states with different consequences. Later work on chat-based tutoring makes the same point empirically, finding that resolved confusion predicted better post-test performance while frustration did not (Gallo and Stein, 2023).

2.2 Productive struggle

In the learning sciences, some failure is desirable. Productive failure designs deliberately let learners attempt problems before instruction (Kapur, 2008). For our purposes, the key implication is that an assistant should not try to remove all negative affect. The question is whether a student is still making progress while frustrated.

2.3 Affective design

Norman distinguishes visceral, behavioral and reflective levels of emotional response to designed things (Norman, 2004). A failed run hits all three: the red error text (visceral), the broken flow of work (behavioral), and the student's sense of whether they are "good at this" (reflective). Broaden-and-build theory proposes that positive emotions broaden the range of thoughts and actions a person considers (Fredrickson, 2001); frustrated students, by contrast, tend to narrow to retrying the same fix. Design work on recovery after failure catalogues patterns such as naming the next smallest step and making retries cheap (Whitfield, 2021).

3 Related Work

3.1 Detecting frustration in text

Albrecht and Nwosu built a lexicon of 214 frustration markers in text-based algebra tutoring, grouped into six categories (repetition with negation, terse replies, explicit giving up, capitalization and punctuation, self-deprecation, and complaints about the tutor), and validated marker counts against self-reports (Albrecht and Nwosu, 2022). Their strongest marker was repetition of a prior question with added negation. We adopt their categories as a starting point.

3.2 Confusion versus frustration

Gallo and Stein labeled confusion and frustration separately in chat tutoring logs and showed that merging them into a single "negative affect" class causes interventions to fire on confused students who would have resolved the impasse on their own (Gallo and Stein, 2023). We therefore code both states and allow a turn to carry both.

3.3 Responding to frustration

In a Wizard-of-Oz study, tutors that acknowledged frustration and reframed the difficulty increased persistence after failure, while apologies did not and reduced perceived competence (Yamada and Ortiz, 2024). Repeated empathic phrases were read as hollow. These findings inform our design implications in Section 7.

3.4 Programming assistants

Work on how novices use code assistants describes shifts from treating the assistant as a sounding board to treating it as an oracle after repeated failures (Lindgren and Mensah, 2025). That shift is one of the behaviors we expected to see around frustration episodes.

4 Data and Method

4.1 Data

We analyzed 180 de-identified conversations between CS1 students and an AI programming assistant, collected in an IRB-approved lab study in which students completed two debugging tasks with the

assistant available. Students consented to analysis of their conversation text for research. Conversations were linked to the run history of the student's code (timestamps and pass/fail) but not to any grades or identities. We received the data stripped of names and with code identifiers hashed.

Property	Value
Conversations	180
Distinct students	94
Student turns	2,912
Assistant turns	2,870
Median turns per conversation (student)	14
Median conversation length	11.5 minutes
Conversations ending with a passing run	121 (67%)

4.2 Sampling for coding

Jordan built a sampling script that stratified conversations by outcome (passed or not) and length, so that long, unsuccessful conversations, where frustration was most likely, were not under-represented. All 180 conversations were coded in full.

4.3 Coding procedure

Maya adapted the lexicon categories to programming help (Section 5). David built a small web tool that showed each student turn with the two preceding turns and the run history, and recorded codes. Each turn could be coded frustrated, confused, both, or neither. Two coders (David and Maya) independently coded a calibration set of 20 conversations (318 turns), discussed disagreements, revised the codebook, and then independently coded a reliability set of 30 conversations (502 turns). Cohen's kappa on the reliability set was 0.68 for frustration and 0.63 for confusion, which we consider substantial and moderate agreement respectively (Cohen, 1960; Landis and Koch, 1977). The remaining conversations were split between the two coders.

4.4 Detectors

We compared three detectors of frustration at the turn level, each evaluated with five-fold cross-validation grouped by student so that no student appeared in both training and test folds:

• Marker counts: counts of each codebook category in the turn and in the previous two student turns, fed to logistic regression.

• Baseline-normalized markers: the same counts plus each turn's reply length and response latency expressed as z-scores relative to the same student's earlier turns (Jordan's idea, motivated by the finding that terse replies are ambiguous across people).

• Zero-shot LLM: a general-purpose language model prompted with our codebook definitions and asked to label each turn, with the same two-turn context.

5 Codebook

Our final codebook has seven frustration markers and four confusion markers. Table 2 lists them with examples paraphrased from the data (we do not quote students verbatim).

Marker	State	Example (paraphrased)	Share of frustrated turns
Repeat with negation	Frustration	"no, it still gives the same error"	34%
Paste-same-error	Frustration	the identical traceback pasted a second time	27%
Terse reply (relative to own baseline)	Frustration	"ok" after a long explanation	22%
Explicit giving up	Frustration	"never mind I'll just submit it"	11%
Complaint about the assistant	Frustration	"you keep giving me the same code"	14%
Emphatic typography	Frustration	all caps, repeated question marks	9%
Self-deprecation	Frustration	"I'm so bad at this"	6%
Hedged question with "but"	Confusion	"but why does the loop stop early?"	-
Restating the assistant	Confusion	student rephrases the explanation as a question	-
Asking for an example	Confusion	"can you show what that means with a list"	-
Explicit not-understanding	Confusion	"I don't get what a reference is"	-

Two changes from the original lexicon proved important. First, we added paste-same-error, which has no counterpart in algebra tutoring: a student who pastes the same traceback twice is signalling that the assistant's previous answer did not help. Second, we made terse replies relative to each student's own typical reply length, because some students always write short messages. Self-deprecation was rare and often joking, as Albrecht and Nwosu also observed, so we kept it but it carried little weight.

6 Results

6.1 Prevalence (RQ1)

Of 2,912 student turns, 412 (14.1%) were coded frustrated, 538 (18.5%) confused, and 97 (3.3%) both. Frustration was concentrated: 48 of 180 conversations (27%) contained at least three frustrated turns, and these accounted for 71% of all frustrated turns.

Confusion and frustration looked different in text. Confused turns were longer (median 19 words vs 9) and more often contained a question about why something happened. Frustrated turns were shorter, more often contained pasted code or tracebacks, and more often asked for a fix rather than an explanation.

6.2 When frustration appears (RQ2)

We aligned each conversation with the run history. The clearest pattern involved failed runs that followed an assistant suggestion. The probability that a student's next turn was frustrated was 6% after a passing run, 13% after one failed run following an assistant suggestion, and 31% after the second consecutive failed run following an assistant suggestion.

Preceding event	Next student turn frustrated
Passing run	6%
Failed run, no assistant suggestion applied	10%
1st failed run after applying a suggestion	13%
2nd consecutive failed run after suggestions	31%

3rd or later	38%

The assistant behavior most often preceding frustration was returning a full replacement code block that changed parts of the program the student had not asked about (present before 44% of frustrated turns versus 19% of other turns). Students described this as the assistant "rewriting everything". Frustration also followed explanations that repeated a previous explanation nearly verbatim.

Confusion that was followed within two turns by an explanation the student acknowledged ("oh that makes sense") rarely turned into frustration (8%). Unresolved confusion turned into frustration within five turns in 29% of cases, consistent with the impasse account of affect dynamics.

Outcome mattered: conversations with three or more frustrated turns ended in a passing run 44% of the time, compared with 76% for the rest. Several of these ended with an explicit giving-up turn.

6.3 Detection (RQ3)

Detector	AUC	F1 (frustrated)	Precision	Recall
Marker counts (LR)	0.74	0.49	0.46	0.53
Baseline-normalized markers (LR)	0.79	0.55	0.54	0.57
Zero-shot LLM with codebook	0.71	0.47	0.39	0.60

Normalizing against each student's baseline improved every metric. The zero-shot model over-labeled frustration: it treated most terse replies and most pasted errors as frustrated, regardless of what the student usually did. It was also sensitive to wording in the prompt; small changes to the definition moved its F1 by up to 0.06. The logistic models are less impressive in absolute terms, but they are interpretable: the largest coefficients were for repeat-with-negation, paste-same-error and the relative-terseness feature.

6.4 Error analysis

False positives of the normalized detector were mostly students who were brisk but not upset, often late in a successful conversation ("ok thanks", "works"). False negatives were mostly polite frustration ("sorry, I tried that and it still fails, could you check again?"), which contains no markers apart from the repeat. These cases suggest adding run-history features, which we did not do in order to keep the detector text-only.

7 Design Implications

Our results suggest that an assistant can notice likely frustration from text it already has, especially when it compares a student against their own earlier turns. The more important question is what it should do next.

7.1 Change strategy, not just wording

The strongest precursor of frustration was a sequence of failed runs after suggestions. When an assistant sees a repeated error after its own fix, sending another full code block repeats the pattern. Following recovery patterns from affective design (Whitfield, 2021), the assistant could name the next smallest step ("let's check just what the list contains before the loop"), or ask the student to run one targeted print statement.

7.2 Acknowledge and reframe, briefly

Acknowledging frustration with information about why the problem is hard increased persistence in prior work, while apologies did not (Yamada and Ortiz, 2024). An assistant might say that off-by-one errors in loops are among the most common bugs in the course, then propose a check. The acknowledgement should not repeat; a fixed empathy script will read as hollow by the second time.

7.3 Protect productive confusion

Because resolved confusion was rarely followed by frustration, an assistant should not treat confusion as something to shut down with a complete answer. A detector that merges the two states would trigger help at the wrong moment (Gallo and Stein, 2023).

7.4 Hand off to a person

For the 27% of conversations with sustained frustration, the right response may be a human. An assistant could suggest office hours or flag the session (with consent) for a TA check-in rather than continue generating code.

7.5 Visceral design of errors

At the visceral level, red tracebacks after each assistant suggestion amplify a sense of repeated failure. Showing which part of the error changed since the last run would make progress visible even when the run fails.

8 Limitations

Our data come from a lab study with two debugging tasks, not from students' own assignments under deadline pressure, where frustration may look different. We coded frustration from text, without self-reports, so our labels are an observer's judgment. Agreement for confusion (kappa 0.63) was lower than we would like. The 180 conversations came from one course and one assistant. We did not test any of the design implications; they are hypotheses for future work. Finally, our zero-shot comparison used a single model and prompt family and should not be read as a general claim about language models.

9 Conclusion

Frustration in student-assistant conversations is common, concentrated in a minority of conversations, and readable from text. It follows a recognizable sequence: an assistant's fix fails, then fails again, and the student repeats themselves with growing negation. A simple, interpretable detector that compares students against their own baseline does better than a zero-shot model. The design challenge is to respond to that signal by changing strategy, acknowledging difficulty briefly and honestly, protecting productive confusion, and knowing when to hand the student to a person.

Team Contributions

David Barron: data access and IRB amendment, coding tool, coding (50%), detector experiments, Sections 1, 4 and 6. Maya Torres: codebook adaptation, coding (50%), reliability analysis, Sections 2, 3 and 5. Jordan Lee: sampling script, baseline-normalized features, error analysis, Sections 7 and 8, presentation slides. All authors edited the full report.

References

Albrecht, Nwosu (2022). Frustration Markers in Text-Based Tutoring Dialogues: A Lexicon and Validation Study. International Journal of Artificial Intelligence in Education, 32(4), 901-930.

Baker, D'Mello, Rodrigo et al. (2010). Better to Be Frustrated than Bored: The Incidence, Persistence, and Impact of Learners' Cognitive-Affective States during Interactions with Three Different Computer-Based Learning Environments. International Journal of Human-Computer Studies, 68(4), 223-241.

Cohen (1960). A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement, 20(1), 37-46.

D'Mello, Graesser (2012). Dynamics of Affective States during Complex Learning. Learning and Instruction, 22(2), 145-157.

Fredrickson (2001). The Role of Positive Emotions in Positive Psychology: The Broaden-and-Build Theory of Positive Emotions. American Psychologist, 56(3), 218-226.

Gallo, Stein (2023). Confusion Is Not Frustration: Distinguishing Epistemic Affect in Chat-Based Tutoring Logs. Proceedings of the 16th International Conference on Educational Data Mining, 77-88.

Kapur (2008). Productive Failure. Cognition and Instruction, 26(3), 379-424.

Landis, Koch (1977). The Measurement of Observer Agreement for Categorical Data. Biometrics, 33(1), 159-174.

Lindgren, Mensah (2025). "It's Like a TA That Never Sighs": Novices' Mental Models of Code Assistants in CS1. Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1, 88-94.

Norman (2004). Emotional Design: Why We Love (or Hate) Everyday Things. Basic Books.

Picard (1997). Affective Computing. MIT Press.

Russell (1980). A Circumplex Model of Affect. Journal of Personality and Social Psychology, 39(6), 1161-1178.

Whitfield (2021). Designing for Recovery: Affective Design Patterns After Failure in Learning Software. Proceedings of the 2021 ACM Designing Interactive Systems Conference, 1250-1262.

Yamada, Ortiz (2024). Apologies, Empathy, and Persistence: How Conversational Tutors' Affective Responses Shape Learner Effort. Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems.